

VAJRA

Deployment and Operations Guide

Installing and running the studio on a Linux GPU host

Applies to	VAJRA 2.5 (branch vajra-2.5)
Target	Linux host with an NVIDIA GPU. Not Colab.
Archive	VAJRA_handover_<date>.zip — source only, ~0.3 MB
Weights	Downloaded during installation, or supplied separately

This guide assumes no prior knowledge of the system. It covers checking the machine, installing, running, what each module costs, and what to do when something fails. The troubleshooting section describes faults that actually occurred during development rather than hypothetical ones.

1. What you received, and what you did not

The archive contains **source code only**, around 0.3 MB. Three things are deliberately absent, and each is rebuilt or re-downloaded during installation:

Absent	Why	Restored by
Virtual environments venvs/, venv_main/	Not portable. They hardcode absolute paths and are built against a specific interpreter and CUDA runtime. Copied to another machine they look installed and fail at import.	install.sh, in minutes
Model weights models/	Tens to hundreds of gigabytes, and freely re-downloadable.	install.sh, or copied in (\$7)
Third-party sources third_party/	Cloned from upstream at fixed versions.	install.sh

The archive also carries **MANIFEST.txt** with a SHA-256 for every file. Verify it before installing — a truncated archive that still opens is a real failure mode when moving files between institutions.

2. Requirements

Hardware

Component	Minimum	Notes
GPU	NVIDIA, 16 GB VRAM	24 GB+ comfortable. The video-generation modules want far more; the core modules run in 16 GB.
Disk	60 GB free	Core modules only. See \$5 for the full accounting — the two video models exceed 100 GB <i>each</i> .
RAM	32 GB	Models are staged through system memory on load.

Software

Two Python interpreters are required. This is the least obvious prerequisite and the most common cause of a failed install. The voice and lip-sync stacks depend on packages that publish no wheels for Python 3.12, so they are built from 3.10, while everything else runs on 3.12.

```
sudo apt update
sudo apt install -y python3.12 python3.12-venv python3.12-dev \
python3.10 python3.10-venv python3.10-dev \
ffmpeg git git-lfs util-linux build-essential
```

An NVIDIA driver supporting CUDA 12.x must already be installed; check with `nvidia-smi`. CUDA toolkits are not needed separately — each environment installs its own PyTorch build.

3. Installation

Step 1 — unpack and verify

```
unzip VAJRA_handover_<date>.zip
cd VAJRA
sha256sum -c <(awk '/^  [0-9a-f]{64}/ {print $1" "$2}' MANIFEST.txt)
```

Step 2 — check the machine BEFORE installing anything

```
python3 setup/preflight.py
```

This reports the GPU, both interpreters, ffmpeg, free disk, privileges and network reachability. Each failure says what is missing and the command to fix it. Run it to completion before continuing — every problem it catches is one that would otherwise surface part-way through a multi-gigabyte download.

Step 3 — install

```
bash install.sh                # core modules
bash install.sh voice lipsync viitor # add tier A voice editing
bash install.sh all             # everything (250 GB+)
```

With no arguments it installs the core modules: Video Edit, Image Generation, Face Swap and Media Studio. Installation is **resumable** — re-running skips whatever already exists, so a module can be added later without rebuilding what is present.

Note. Weights are stored under `models/` by default. If several people share the host, set `VAJRA_MODELS=/shared/vajra-models` before installing so they are fetched once and reused. Environments are always local to the installation.

Step 4 — run

```
bash run.sh                # http://localhost:7860
VAJRA_PORT=8080 bash run.sh # different port
VAJRA_SHARE=1 bash run.sh  # also print a public link
```

Important. Sharing is **off** by default on a host installation. Enabling it publishes a public URL to the interface with no authentication, for anyone who has the link. Do not enable it on a machine holding sensitive material.

Step 5 — confirm the installation is sound

```
venv_main/bin/python tests/test_dsp.py
venv_main/bin/python tests/test_notebook.py
```

These run on CPU in seconds and need no model weights. They exercise the numerical core and the setup wiring. A full list is in `tests/README.md`.

4. The modules

Module	What it does	Environments	Weights
Video Edit	Edit a talking-head video's transcript; only the changed words are re-voiced and only the affected frames re-synced.	voice, lipsync (+ demucs)	voice, lipsync
Voice Generation	Regenerate changed words in place, or speak new text in a cloned voice. English only.	viitor	viitor

Module	What it does	Environments	Weights
Image Generation	Photoreal images from a prompt.	principal runtime	on first use
Face Swap	A source face onto a target image or video.	principal runtime	faceswap
Text → Video	Video from a prompt, or animate a photo.	ltx2 and/or wan	on first use
Image Edit	Edit 1–3 images by instruction.	qwen	on first use
Media Studio	Trim, convert, caption, remove backgrounds. CPU only.	principal runtime	small, on first use

Each isolated environment exists because these model families pin mutually incompatible versions of the same core libraries — no single environment can host them all. A side benefit: a subprocess returns its GPU memory unconditionally when it exits.

5. Disk footprint

Measured from the model repositories, not estimated. These are what is actually fetched with the shipped configuration.

Module	Models	Download
Video Edit	Transcription 3.09 · voice clone 2.09 · lip-sync 9.82 · fallback 0.44 · separation 0.08	15.5 GB
Voice Generation	Three models: codec, language model, aligner	11.6 GB
Image Generation	One diffusion model	13.9 GB
Face Swap	Four small models	1.6 GB
Media Studio	Background removal	0.2 GB
Core subtotal	everything above	42.8 GB
Text → Video	Two engines, distributed at full precision	88.5 + 126.2 GB
Image Edit	One editing model	57.7 GB

Important. The two video-generation models are **each larger than a 112 GB disk**. Install them one at a time, never alongside another heavy module. The build refuses up front if there is not enough space rather than failing part-way through the download.

6. Operating notes

One model on the GPU at a time

The interface serves requests concurrently but there is one accelerator. A process-wide lock serialises all GPU work, and loading a model explicitly evicts the incumbent. This trades throughput for reliability deliberately: a long render cannot be starved of memory by an unrelated request arriving mid-run.

Where things are written

Path	Contents
outputs/	Every generated artefact, timestamped, prefixed vajra_
models/ or \$VAJRA_MODELS	Model weights; safe to share between installations
venvs/, venv_main/	Environments; never copy these between machines
third_party/	Upstream sources cloned at install time

Running as a service

For an always-on deployment, a systemd unit is the simplest approach:

```
[Unit]
Description=VAJRA
After=network.target

[Service]
User=vajra
WorkingDirectory=/opt/VAJRA
Environment=VAJRA_MODELS=/shared/vajra-models
ExecStart=/bin/bash /opt/VAJRA/run.sh
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

7. Air-gapped installation

If the host cannot reach the internet, preflight will say so. Two things must then be brought in from a machine that can:

1. Model weights. On a connected machine, install and let it download, then copy the model directory across and point the offline host at it:

```
# connected machine
bash install.sh voice lipsync
tar czf vajra-models.tar.gz -C models .

# offline host
mkdir -p /shared/vajra-models
tar xzf vajra-models.tar.gz -C /shared/vajra-models
export VAJRA_MODELS=/shared/vajra-models
```

Alternatively, build the archive with weights included: `python3 tools/make_handover.py --with-weights`.

2. Python packages. The environments install from PyPI. Mirror them on the connected machine and point pip at the copy:

```
pip download -r requirements/main.txt -d wheels/
# on the offline host
export PIP_NO_INDEX=1 PIP_FIND_LINKS=/path/to/wheels
```

Note. Third-party sources are cloned from GitHub by `install.sh`. On an air-gapped host, copy the `third_party/` directory from a connected installation instead; `install.sh` skips cloning whatever is already present.

8. Troubleshooting

Every entry below is a fault that actually occurred, with the cause rather than only the symptom.

Symptom	Cause and remedy
No such file or directory: <code>.../python_discovery/_cache.py</code>	Two environment builds writing to the same system site-packages at once. Fixed in this version by a lock; if it recurs, build one target at a time.
ENOSPC / no space left	The disk filled mid-download. Check \$5 before building; set <code>VAJRA_MODELS</code> to a larger volume.
Transcript comes back in the wrong language or script	Language auto-detection runs on the first ~30 seconds and mistakes accented English for a related language, then transliterates. Set Spoken language explicitly in the Video Edit tab.

Symptom	Cause and remedy
Synthesis fails with NotImplementedError	Older builds crashed on any digit in some languages. Fixed here; if seen, confirm you are running 2.5 or later.
“DURATION CAPPED ... off by N seconds”	The new text needs far more time than the audio it replaces, and the track must stay frame-aligned with the video. Change fewer words per line, keep the replacement near the original syllable count, or enable Replace the entire track , which re-times the picture.
Voice Generation: 503, connection refused	Its five services had not all started. The first start loads three models and takes several minutes. If it persists: <code>bash run_grpc_v2.sh logs encoder inside third_party/vitor-voice-nar</code> .
Lip-sync output looks soft or wrong	Usually a stale checkpoint paired with a newer config. Delete <code>models/latentsync</code> and re-run the weight download.
Interface starts, every generate button errors	That module's environment or weights were not installed. The error names which. Re-run <code>install.sh</code> with that target.

9. Known limitations

Stated plainly so they are not discovered as surprises:

- **Voice Generation is English only.** Other languages fall back to phrase-level regeneration, which the interface reports as a lower tier.
- **Editing cannot lengthen speech much.** Replacement audio must occupy the slot it replaces to stay frame-aligned. Beyond roughly 15% the stretch becomes audible; past that the whole-track retake is the only honest option, and it re-times the picture.
- **The detectability scorecard measures joins, not quality.** It reports whether an edit's seams stand out against the recording's own word boundaries. Speaker similarity, naturalness, intelligibility and lip-sync accuracy are named as unmeasured rather than guessed at.
- **Lip-sync prefers clean frontal footage.** Profiles, occlusion, poor light and multiple speakers in frame all degrade it.
- **Video-generation modules are impractical on a small disk.** See §5.

10. Getting further

Reference	Where
Module and architecture overview	README.md
What the offline tests cover, and what they do not	tests/README.md
Why each design decision was made	Source comments — the reasoning is recorded alongside the code rather than separately
Architecture diagrams	VAJRA_Architecture_Diagrams.pptx, with detailed speaker notes on every figure